

Assignment 1 Custom IP and Simulation

Tianze Zhang
1005459585

Simple Mask Module and Testbench

Q1. A description of why you chose the specific test inputs you used in your Verilog testbench:

Answer:

I used the testbench to implement eight distinct tests:

First, I incorporated the default test given in the Assignment 1 handbook, guaranteeing that the findings are immediately verified against the manual's standards.

In addition, I created a random decimal number. This helps me to ensure that the output values are in binary and hexadecimal format.

finally, I used six extreme conditions:

- masking all 31 bits with input value of 1
- masking zero bits with input value of 1
- masking all 31 bits with input value of 0
- masking zero bits with input value of 0
- masking all 31 bits with input value of 32
- masking zero bits with input value of 32

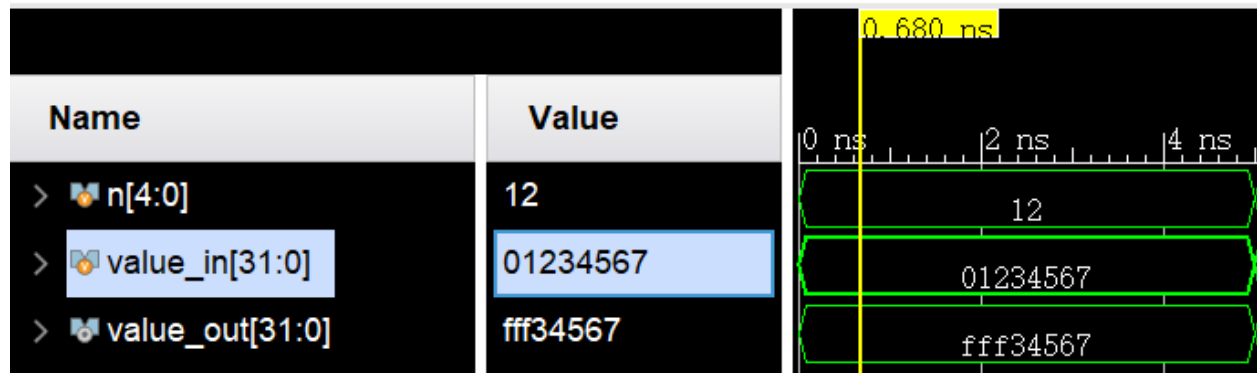
These tests fully assess the system's functionality under a variety of circumstances.

Q2. A summary of the output of each of the simulation methodologies (can be displayed as a figure)

Answer:

Default test given in the Assignment 1 handbook

Waveform:



Console output using display:

```

| TEST 1 START (example in manual)
| value_in(binary)  = 00000001001000110100010101100111,
| value_out(binary) = 1111111111100110100010101100111,
| value_out(hex)    = fff34567
| Test 1 Passed
| END OF TEST 1
| //-----

```

Console output using monitor:

```

monitor:
n = 12,
value_in = 01234567,
value_out = fff34567
//-----

```

Random decimal number as a regular testing:

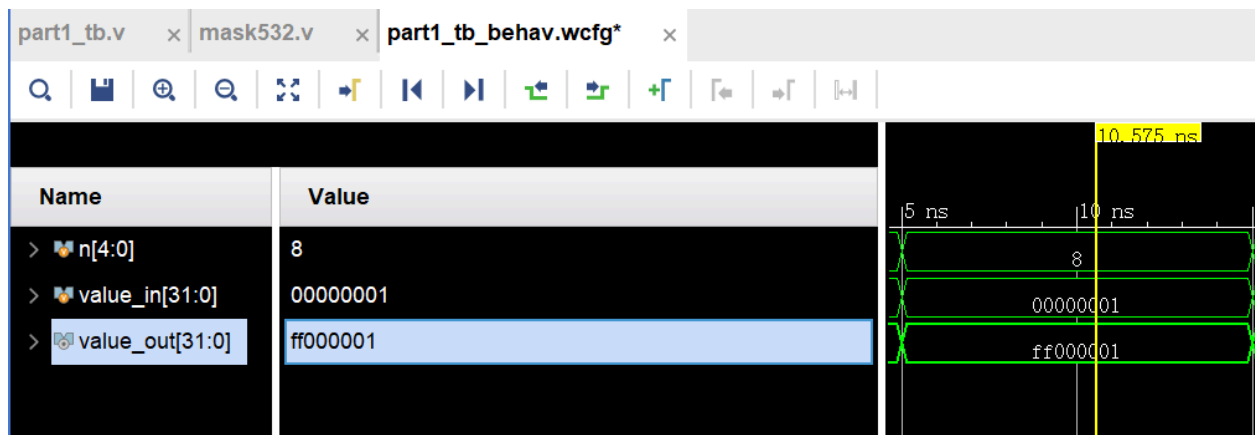
```

monitor:
n = 8,
value_in = 00000001,
value_out = ff000001

display method:
TEST 2 START
value_in(binary) = 00000000000000000000000000000001,
value_out(binary) = 11111111000000000000000000000001,
value_out(hex) = ff000001
Test 2 Passed
END OF TEST TWO

```

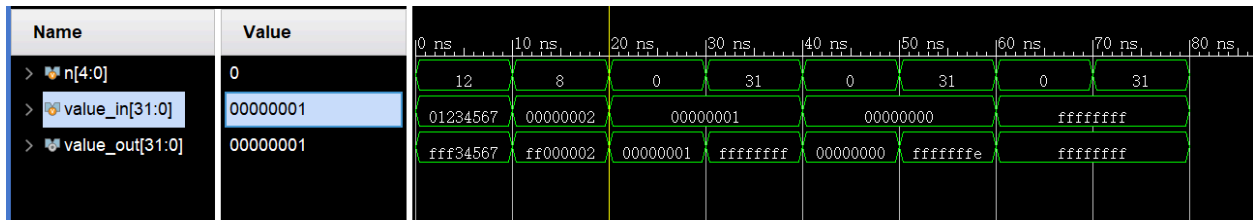
Waveform:



Extreme conditions:

- masking all 31 bits with input value of 1
- masking zero bits with input value of 1
- masking all 31 bits with input value of 0
- masking zero bits with input value of 0
- masking all 31 bits with input value of 32
- masking zero bits with input value of 32

waveform :



Monitor:

```
//-----  
monitor:  
n = 0,  
value_in = 00000001,  
value_out = 00000001  
n = 31,  
value_in = 00000001,  
value_out = ffffffff  
n = 0,  
value_in = 00000000,  
value_out = 00000000  
n = 31,  
value_in = 00000000,  
value_out = ffffffff  
n = 0,  
value_in = ffffffff,  
value_out = ffffffff  
n = 31,  
value_in = ffffffff,  
value_out = ffffffff
```

Q3. A discussion comparing the different simulation methods. What are the advantages and disadvantages of each? Are there specific situations in which one would be preferable to the others?

Answer:

The provided diagram discusses the pros, cons, and ideal situations for three methods of signal behavior analysis: Waveform, Display, and Monitor.

Waveform:

Pros:

- Provides a thorough representation of signal behaviour.
- Helps to identify time analysis and signal transitions.
- Easy access; no additional testbench code is required.

Cons:

- Requires manual examination, which might be difficult with a high number of signals
- Only one radix may be exhibited at a time, making the observation less apparent.
- Debugging solely combinational designs takes longer than text-based output.

Ideal Situation:

- Best suited for extensive signal behaviour analysis with a focus on time.

Display (\$Display):

Pros:

- Provides instant feedback on test findings, enabling speedy verification.
- Ideal for reporting intermediate values in loops and computations.

Cons:

- Does not offer specific information on signal behaviour and is restricted to textual output.
- Manually writing the test and condition may be time-consuming if a big volume of testing is necessary.
- Variables are not tracked continually; only when called are they printed.
- In comparison to waveforms, timing errors are more difficult to troubleshoot.

Ideal Situation:

- Suitable for quickly verifying test results and detecting test faults.
- Printing ultimate outputs or significant intermediate findings.

Monitor (\$Monitor):

Pros:

- Provides real-time monitoring of variable selected.
- tracking variable changes during the simulation.

Cons:

- To avoid excessive output, carefully decide which variables to monitor.
- combinational logic propagation delays may not be counted.

Ideal Situation:

- Effective for real-time monitoring of signal readings and tracking variable changes during simulation.

Creating an AXI IP

Q1. A short description of how you implemented the required change in the AXI-Lite slave register code which ensured that the `init_write` signal stayed set for only a single cycle when written

Answer:

The `init_write` signal is used to trigger actions on the AXI-Lite master interface, namely initiating a write operation. To ensure that `init_write` remains active for only one cycle, it is reset after its conditions have been evaluated and used for the intended operations within the same cycle. This mechanism is implemented in different always blocks as follows:

Initiating Write Address (`M_AXI_AWADDR`) and Data (`M_AXI_WDATA`):

- The signal `init_write` is checked in the always block that handles the master interface address channel (`M_AXI_AWADDR` and `M_AXI_AWVALID`).
- When `init_write` is high, and `M_AXI_AWVALID` is not yet asserted, the address is set (`M_AXI_AWADDR <= output_addr`), and `M_AXI_AWVALID` is asserted (`M_AXI_AWVALID <= 1'b1`).
- Importantly, immediately after setting these signals, `init_write` is cleared (`init_write <= 1'b0`). This ensures that the `init_write` action does not extend beyond a single cycle, preventing repeated initiations of the write operation.

```

184 // AXI-Lite Master Interface Logic
185 // Addr Data write interface
186 always @(posedge aclk) begin
187     if (!aresetn) begin
188         M_AXI_AWVALID <= 1'b0;
189     end else begin
190         // "The master can assert the AWVALID signal only when it drives valid address and control information. When
191         // asserted, AWVALID must remain asserted until the rising clock edge after the slave asserts AWREADY" - AXI4_specification
192         // "upon seeing a high value on the single-bit init_write register,
193         // initiates a write on the AXI-Lite master interface" - requirement
194         if (init_write && !M_AXI_AWVALID) begin
195             M_AXI_AWADDR <= output_addr;
196             M_AXI_AWVALID <= 1'b1;
197             init_write <= 1'b0; // Ensure `init_write` is active for only one cycle
198         end
199
200         // Write Address Handshake
201         if (M_AXI_AWVALID && M_AXI_AWREADY) begin
202             M_AXI_AWVALID <= 1'b0;
203         end
204     end
205 end

```

Controlling Data Write Operations:

- Similarly, in the data interface always block, init_write is used to initiate data transfer. The data (M_AXI_WDATA) and its valid signal (M_AXI_WVALID) are set when init_write is high.
- Again, init_write is cleared right after setting these signals. This ensures the operation is limited to a single cycle.

```

208 always @(posedge aclk) begin
209     if (!aresetn) begin
210         M_AXI_WVALID <= 1'b0;
211         // Write Data Handshake
212     end else begin
213         // "upon seeing a high value on the single-bit init_write register,
214         // initiates a write on the AXI-Lite master interface" - requirement
215         if (init_write /*&& !M_AXI_AWVALID*/ && !M_AXI_WVALID) begin
216
217             M_AXI_WSTRB <= 4'b1111;
218             M_AXI_WVALID <= 1'b1;
219
220             M_AXI_AWADDR <= output_addr;
221             M_AXI_WDATA <= masked_value;
222
223             init_write <= 1'b0; // Ensure `init_write` is active for only one cycle
224         end
225
226         // "When asserted, WVALID must remain asserted until the rising clock edge after the slave asserts WREADY." - AXI4_specification
227         if (M_AXI_WVALID && M_AXI_WREADY) begin
228             M_AXI_WVALID <= 1'b0;
229             M_AXI_WSTRB <= 4'b0000;
230         end
231     end
232 end

```


With above mechanism; init_write do not repeat unintentionally, we can verify the behavior of init_witre and master axi interface logic using waveform driven by testbench.



As we can see a high value in the single-bit init_write register triggers a write on the AXI-Lite master interface. Handshaking on the AW signals initiates a write, handshaking on the W signals sends the data, and properly receives the B write response signals. The init_write signal should only be high for one cycle to begin a single write.

M_AXI_WDATA is correctly showing the right value with masking n value equals to 9 and a input_value of 1, the final result indicated that the functionality of my axi slave and master interface is conformant with the lab manual description.

Q2. A short description of how you implemented the AXI-Lite master writing functionality

Answer:

The AXI-Lite master writing functionality in the provided Verilog module is designed to handle the initiation and completion of write operations from an AXI-Lite slave interface to an AXI-Lite master interface. This involves managing address and data channels along with the necessary handshaking to ensure reliable data transfer according to AXI protocol standards. Here's how the functionality is implemented in detail:

Master Write Address Channel

1. Setting Up Write Address and Validity:

- Within this block, the module checks if the `init_write` signal is set (indicating a request to start a write operation) and if `M_AXI_AWVALID` is not already asserted (indicating that the previous address has been accepted or no address is currently being sent).
- If these conditions are met, the module sets the master interface's write address (`M_AXI_AWADDR`) to `output_addr` (derived from slave inputs) and asserts the `M_AXI_AWVALID` signal to indicate that the address is valid and ready to be processed by the master's receiving end.

2. Address Handshake with Master:

- To complete the address transmission, the code then checks for acknowledgment from the master (`M_AXI_AWREADY`). If the master interface signals that it is ready to receive the address by asserting `M_AXI_AWREADY`, the slave deasserts `M_AXI_AWVALID` in the next clock cycle. This step completes the handshake for the address channel.

```

185 // Addr Data write interface
186 always @(posedge aclk) begin
187     if (!aresetn) begin
188         M_AXI_AWVALID <= 1'b0;
189     end else begin
190         // "The master can assert the AWVALID signal only when it drives valid address and control information. When
191         // asserted, AWVALID must remain asserted until the rising clock edge after the slave asserts AWREADY" - AXI4_specification
192         // "upon seeing a high value on the single-bit init_write register,
193         // initiates a write on the AXI-Lite master interface" - requirement
194         if (init_write && !M_AXI_AWVALID) begin
195             M_AXI_AWADDR <= output_addr;
196             M_AXI_AWVALID <= 1'b1;
197             init_write <= 1'b0; // Ensure `init_write` is active for only one cycle
198         end
199
200         // Write Address Handshake
201         if (M_AXI_AWVALID && M_AXI_AWREADY) begin
202             M_AXI_AWVALID <= 1'b0;
203         end
204     end
205 end

```

Master Write Data Channel

1. Setting Up Write Data and Validity:

- Similar to the address channel, the data channel (M_AXI_WDATA and M_AXI_WVALID) also uses an always block triggered by the clock's positive edge. The code checks for the same init_write signal along with the status of M_AXI_WVALID to prevent overwriting ongoing transmissions.
- Upon a valid init_write, the module sets the write data (M_AXI_WDATA) to masked_value (processed data ready to be written) and asserts M_AXI_WVALID. Additionally, the write strobe (M_AXI_WSTRB) is set to 4'b1111 indicating that all bits of the data are valid for writing.

2. Data Handshake with Master:

- The completion of the data transmission is contingent on the master acknowledging the readiness to accept the data by asserting M_AXI_WREADY. Once M_AXI_WREADY is observed, M_AXI_WVALID is deasserted in the subsequent cycle, marking the end of that data transfer.

```

207 | // Data write interface
208 | always @(posedge aclk) begin
209 |     if (!aresetn) begin
210 |         M_AXI_WVALID <= 1'b0;
211 |         // Write Data Handshake
212 |     end else begin
213 |         // "upon seeing a high value on the single-bit init_write register,
214 |         // initiates a write on the AXI-Lite master interface" - requirement
215 |         if (init_write /*&& !M_AXI_WVALID*/ && !M_AXI_WVALID) begin
216 |
217 |             M_AXI_WSTRB <= 4'b1111;
218 |             M_AXI_WVALID <= 1'b1;
219 |
220 |             M_AXI_WADDR <= output_addr;
221 |             M_AXI_WDATA <= masked_value;
222 |
223 |             init_write <= 1'b0; // Ensure `init_write` is active for only one cycle
224 |         end
225 |
226 |         // "When asserted, WVALID must remain asserted until the rising clock edge after the slave asserts WREADY." - AXI4_specification
227 |         if (M_AXI_WVALID && M_AXI_WREADY) begin
228 |             M_AXI_WVALID <= 1'b0;
229 |             M_AXI_WSTRB <= 4'b0000;
230 |         end
231 |     end
232 | end

```

3. Simulating an AXI module IP

Q1. A short description of your testbench functionality

Answer:

The majority of the testbench setup comes from the tutorial 7 example verification IP testbench. The following adjustments are made to test this specific AXI application. All of the test inputs match what I did for the Part 1 test bench.

Using an `init_write` (the trigger), write `n` value, a `value_in`, and an output address to the `mask532`, then get the output value from the BRAM's base address.

First, I need to set up all the memory address offset for this testbench; part of the offsets are setted by the lab manual, such as the value of `n` and `value_in`; nevertheless, I also need to fix up address for bram and base address based on the datasheet of the ddr4 board

```

49  | //Constants
50  | //AXI mask532 base address and register offsets
51  | //following lab manual
52  | localparam mask532_n_OFFSET = 'h0;
53  | localparam mask532_valuein_OFFSET = 'h4;
54  | localparam mask532_outputaddr_OFFSET = 'h8;
55  | localparam mask532_initwrite_OFFSET = 'hC;
56  |
57  | localparam mask532_BASE_ADDRESS = 'h0004_0000;
58  | localparam BRAM_BASE_ADDRESS = 'h0000_0000;
59  | initial begin

```

Then I need to enter values for each of the registers; the value for each register will be detailed in the following question. Finally, the final masked value will be printed to the tcl console using the write function.

```

//test1 is the example in lab manual
$write("----TEST1 begin ----\n");
$write("n_value=12, value_in=0x01234567 \n");
writeRegister(mask532_BASE_ADDRESS + mask532_n_OFFSET, 'd5);
writeRegister(mask532_BASE_ADDRESS + mask532_valuein_OFFSET, 'h01234567);
writeRegister(mask532_BASE_ADDRESS + mask532_outputaddr_OFFSET, BRAM_BASE_ADDRESS);
writeRegister(mask532_BASE_ADDRESS + mask532_initwrite_OFFSET, 'h1);
// Wait for all writes to complete
agent.wait_drivers_idle();
// read from BRAM
$write("Masked value from BRAM \n");
agent.wait_drivers_idle();
readRegister(BRAM_BASE_ADDRESS, Rdatabeat);

```

Q2. A description of why you chose the specific test inputs you used in your Verilog testbench

I ran three sets of tests, all of which were clones of the part 1 testbench, which I thought was a thorough test of both the mask532 logic and the axi interface logic.

First test:

This test is designed to validate basic operation with a set of standard inputs that are presumably defined in the lab manual. Setting `n_value` to 12 and `value_in` to 0x01234567 verifies the mask532 module's ability to handle common mid-range values. This guarantees that the module properly reads and processes standard values and addresses.

```
//test1 is the example in lab manual
$write("---TEST1 begin ---\n");
$write("n_value=12, value_in=0x01234567 \n");
writeRegister(mask532_BASE_ADDRESS + mask532_n_OFFSET, 'd5);
writeRegister(mask532_BASE_ADDRESS + mask532_valuein_OFFSET, 'h01234567);
writeRegister(mask532_BASE_ADDRESS + mask532_outputaddr_OFFSET, BRAM_BASE_ADDRESS);
writeRegister(mask532_BASE_ADDRESS + mask532_initwrite_OFFSET, 'h1);
```

Second test:

This test uses minimalistic values for `n_value` and `value_in` (8 and 0x00000001, respectively), focusing on edge cases at the lower end of input values. It checks the module's response to near-minimal input values and verifies that the module and interface handle small, but valid, data correctly. Such tests are crucial for detecting issues in handling small values that might be overlooked in more typical test cases.

```
$write("---TEST2 begin ---\n");
$write("n_value=8, value_in=0x00000001 \n");
writeRegister(mask532_BASE_ADDRESS + mask532_n_OFFSET, 'd8);
writeRegister(mask532_BASE_ADDRESS + mask532_valuein_OFFSET, 'h0000_0001);
writeRegister(mask532_BASE_ADDRESS + mask532_outputaddr_OFFSET, BRAM_BASE_ADDRESS);
writeRegister(mask532_BASE_ADDRESS + mask532_initwrite_OFFSET, 'h1);
```

Third test:

The third test involves an extreme case where `n_value` is set to 31 and `value_in` to 0x00000001. This test is designed to push the module's limits, verifying how it handles maximum input values. It ensures that the module can process the full range of input values without error,

particularly at boundary conditions. This is vital for ensuring robustness and reliability under various operational conditions.

```
$write("---TEST2 begin ---\n");
$write("n_value=31, value_in=0x00000001 \n");
writeRegister(mask532_BASE_ADDRESS + mask532_n_OFFSET, 'd31);
writeRegister(mask532_BASE_ADDRESS + mask532_valuein_OFFSET, 'h0000_0001);
writeRegister(mask532_BASE_ADDRESS + mask532_outputaddr_OFFSET, BRAM_BASE_ADDRESS);
writeRegister(mask532_BASE_ADDRESS + mask532_initwrite_OFFSET, 'h1);
```

Q3. A summary of the output of the simulation (can be displayed as a figure)

Result from tcl console:

```
---TEST1 begin ---
n_value=12, value_in=0x01234567
Masked value from BRAM
BRAM value: 0x00000000
---TEST2 begin ---
n_value=8, value_in=0x00000001
Masked value from BRAM
BRAM value: 0xff000001
---TEST2 begin ---
n_value=31, value_in=0x00000001
Masked value from BRAM
BRAM value: 0xffffffff
```

Waveform:

